HTTP RPC API 구조 분석

개요

IPFS Kubo의 HTTP RPC API는 클라이언트가 IPFS 노드와 상호작용할 수 있는 RESTful 인터페이스를 제공합니다. 이 API는 `/api/v0` 경로에 마운트되며, 모든 CLI 명령어를 HTTP 엔드포인트로 노출합니다.

API 아키텍처

기본 구조

```
HTTP 요청 → HTTP 서버 → 명령어 라우터 → 명령어 실행 → 응답 생성 → HTTP 응답
```

핵심 컴포넌트

1. HTTP 서버 (core/corehttp)

HTTP 서버는 `core/corehttp` 패키지에서 구현되며, 다음과 같은 주요 기능을 제공합니다:

- **서버 설정**: `ServeOption` 인터페이스를 통한 모듈식 서버 구성
- **CORS 처리**: 크로스 오리진 요청 지원
- **인증**: API 키 기반 접근 제어
- **버전 검사**: 클라이언트-서버 버전 호환성 확인

```
// ServeOption은 HTTP 핸들러를 등록하는 함수 타입
```

```
type ServeOption func(*core.IpfsNode, net.Listener, *http.ServeMux) (*http.Serve
```

2. 명령어 시스템 (core/commands)

모든 IPFS 명령어는 `cmds.Command` 구조체로 정의되며, 다음 요소들을 포함합니다:

```
type Command struct {
Helptext HelpText // 도움말 텍스트
Arguments []Argument // 명령어 인자
Options []Option // 명령어 옵션
Run  func(...) // 실행 함수
Subcommands  map[string]*Command // 하위 명령어
Encoders EncoderMap // 출력 인코더
}
```

3. 루트 명령어 구조

`core/commands/root.go` 에서 정의된 루트 명령어는 모든 IPFS 명령어의 진입점입니다:

```
var  Root = &cmds.Command{
Helptext: cmds.HelpText{
Tagline: "Global p2p merkle-dag filesystem.",
Synopsis: "ipfs [options] <command> ...",
},
Options: []cmds.Option{
// 전역 옵션들
```

```
},

}
```

HTTP 엔드포인트 매핑

URL 구조

모든 API 엔드포인트는 다음 패턴을 따릅니다:

```
POST /api/v0/<command>[/<subcommand>]?<options>
```

명령어-엔드포인트 매핑

| CLI 명령어 | HTTP 엔드포인트 | 설명 |

|----------|----------|------|

| `ipfs add <file>` | `POST /api/v0/add` | 파일 추가 |

| `ipfs cat <hash>` | `POST /api/v0/cat?arg=<hash>` | 콘텐츠 조회 |

| `ipfs id [peer]` | `POST /api/v0/id?arg=[peer]` | 피어 정보 조회 |

| `ipfs dag get <cid>` | `POST /api/v0/dag/get?arg=<cid>` | DAG 노드 조회 |

| `ipfs pin add <hash>` | `POST /api/v0/pin/add?arg=<hash>` | 핀 추가 |

하위 명령어 처리

하위 명령어는 URL 경로로 표현됩니다:

```
var rootSubcommands = map[string]*cmds.Command{
```

```
"add": AddCmd,

"cat": CatCmd,

"dag": dag.DagCmd, // dag.DagCmd는 하위 명령어들을 포함

"pin": pin.PinCmd, // pin.PinCmd는 하위 명령어들을 포함

// ...

}
```

요청 처리 과정

1. HTTP 요청 수신

```
func  commandsOption(cctx oldcmds.Context, command *cmds.Command) ServeOption {

return  func(n *core.IpfsNode, l net.Listener, mux *http.ServeMux) (*http.ServeM

cfg := cmdsHttp.NewServerConfig()

cfg.APIPath = APIPath  // "/api/v0"

cmdHandler := cmdsHttp.NewHandler(&cctx, command, cfg)

mux.Handle(APIPath+"/", cmdHandler)

return  mux, nil

}

}
```

2. 요청 파싱 및 검증

HTTP 요청이 수신되면 다음 단계를 거칩니다:

1. **URL 파싱**: `/api/v0/add` → `["add"]` 경로 추출
2. **명령어 해결**: 경로를 통해 해당 명령어 구조체 찾기
3. **인자 추출**: URL 쿼리 파라미터에서 `arg` 값들 추출
4. **옵션 파싱**: 쿼리 파라미터를 명령어 옵션으로 변환
5. **바디 처리**: POST 데이터 (파일 업로드 등) 처리

3. 명령어 실행

```
// 명령어 실행 함수의 일반적인 구조
Run: func(req *cmds.Request, res cmds.ResponseEmitter, env cmds.Environment) err
// 1. API 인스턴스 획득
api, err := cmdenv.GetApi(env, req)
if err != nil {
return err
}
// 2. 요청 파라미터 처리
// 3. 비즈니스 로직 실행
// 4. 결과 응답
return cmds.EmitOnce(res, result)
}
```

4. 응답 생성

응답은 `cmds.ResponseEmitter` 를 통해 생성되며, 다음과 같은 형태를 가집니다:

- **JSON 응답**: 기본 응답 형식
- **스트리밍 응답**: 대용량 데이터나 실시간 데이터
- **바이너리 응답**: 파일 다운로드 등

인증 및 보안

API 키 인증

```
func withAuthSecrets(authorizations map[string]rpcAuthScopeWithUser, next http
return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
authorizationHeader := r.Header.Get("Authorization")
auth, ok := authorizations[authorizationHeader]
if ok {
// 허용된 경로 확인
for _, prefix := range auth.AllowedPaths {
if strings.HasPrefix(r.URL.Path, prefix) {
next.ServeHTTP(w, r)
return
}
}
}
http.Error(w, "Access Denied", http.StatusForbidden)
})
```

```
}
```

CORS 설정

```
func addCORSDefaults(c *cmdsHttp.ServerConfig) {
// 로컬호스트 오리진 허용
c.AppendAllowedOrigins(defaultLocalhostOrigins...)
// 브라우저 확장 프로그램 오리진 허용
c.AppendAllowedOrigins(companionBrowserExtensionOrigins...)
// 기본 HTTP 메서드 설정
if len(c.AllowedMethods()) == 0 {
c.SetAllowedMethods(http.MethodGet, http.MethodPost, http.MethodPut)
}
}
```

에러 처리

HTTP 상태 코드 매핑

- **200 OK**: 성공적인 요청
- **400 Bad Request**: 잘못된 요청 파라미터
- **403 Forbidden**: 인증 실패
- **404 Not Found**: 존재하지 않는 리소스
- **500 Internal Server Error**: 서버 내부 오류

에러 응답 형식

```
{
"Message": "error description",
"Code": 0,
"Type": "error"
}
```

성능 최적화

스트리밍 응답

대용량 데이터 전송을 위한 스트리밍 지원:

```
// 스트리밍 응답 예제 (cat 명령어)
return res.Emit(reader) // io.Reader를 직접 스트리밍
```

연결 재사용

HTTP/1.1 Keep-Alive와 HTTP/2 멀티플렉싱을 통한 연결 최적화

압축

gzip 압축을 통한 대역폭 절약

확장성

커스텀 명령어 추가

새로운 명령어는 다음과 같이 추가할 수 있습니다:

```
var MyCmd = &cmds.Command{
Helptext: cmds.HelpText{
Tagline: "My custom command",
},
Run: func(req *cmds.Request, res cmds.ResponseEmitter, env cmds.Environment) err
// 커스텀 로직
return cmds.EmitOnce(res, result)
},
}

// 루트 명령어에 등록
rootSubcommands["mycmd"] = MyCmd
```

미들웨어 패턴

`ServeOption` 함수를 통한 미들웨어 체인 구성:

```
func MyMiddleware() ServeOption {
return func(n *core.IpfsNode, l net.Listener, mux *http.ServeMux) (*http.ServeM
// 미들웨어 로직
return mux, nil
```

```
}

}
```

모니터링 및 로깅

요청 로깅

```
func (c *Context) LogRequest(req *cmds.Request) func() {

rle := &ReqLogEntry{

StartTime: time.Now(),

Active: true,

Command: strings.Join(req.Path, "/"),

Options: req.Options,

Args: req.Arguments,

}

c.ReqLog.AddEntry(rle)

return  func() {

c.ReqLog.Finish(rle)

}

}
```

메트릭 수집

OpenTelemetry를 통한 HTTP 요청 메트릭 수집:

```
cmdHandler = otelhttp.NewHandler(cmdHandler, "corehttp.cmdsHandler")
```

이러한 구조를 통해 IPFS Kubo는 강력하고 확장 가능한 HTTP RPC API를 제공하며, 클라이언트가 모든 IPFS 기능에 프로그래매틱하게 접근할 수 있도록 합니다.